RELEASE ENGINEERING · WINDOWS CLIENTS

Windows Code Signing Playbook

How signing a Windows executable proves who built it, how Windows checks that proof, what certificate expiry breaks, and how to sign from GitLab CI with the key held in an HSM.

What it proves · How Windows checks · Expiry · Getting & monitoring · What clients see · HSM + GitLab CI · Timestamping · TSA unreachable · Installers · Allow-listing · Checking a release

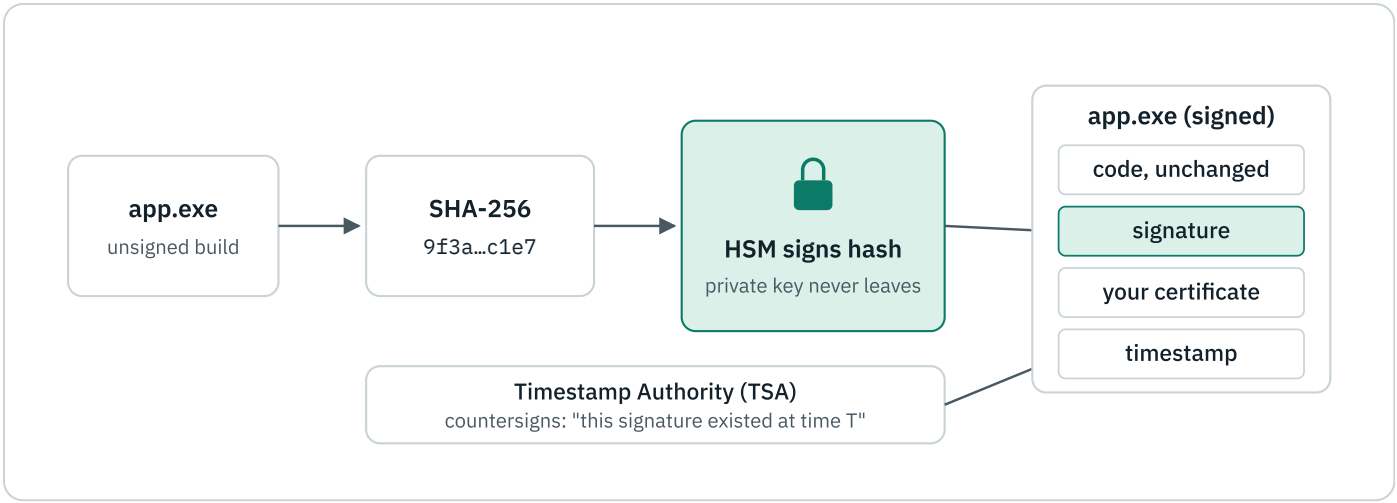
IN SHORT

- A signature proves **who** published the file and that it is **unchanged**. It does not prove the code is safe.
- Always **timestamp** signatures. Then binaries stay valid after the certificate expires.
- **Check the signed file, not the configuration.** A TSA URL in the pipeline doesn't prove the file is timestamped, and the full certificate chain must be embedded. Verify both after signing and fail the job if either is missing.
- Certificates now last at most about 15 months, so renewal is routine. Make the pipeline refuse to sign with an expired certificate.
- Since June 2023, publicly trusted signing keys **must** be kept in certified hardware. The CI job asks the HSM to sign; it never holds the key.
- In managed corporate environments, unsigned or badly signed binaries are usually blocked outright.

---

01 What a code signing certificate proves

Windows signatures use Microsoft's **Authenticode** format. The pipeline hashes the executable, the private key signs that hash, and the signature, your certificate and a timestamp are embedded in the file. The certificate comes from a public Certificate Authority, which checked your organization's legal identity before issuing it.



The certificate ties your public key to your verified company name. Anyone can check the signature with the public key, but only the holder of the private key could have made it.

ASSURANCE	WHAT IT MEANS FOR THE CLIENT
Authenticity	The file was signed by whoever controls the private key, and the CA vouches that this is "Your Company Ltd".
Integrity	No byte changed after signing. Changing even one byte changes the hash, and the signature fails.
Accountability	A real, vetted organization stands behind the binary. Its certificate can be revoked if it is misused.
Not covered	Whether the code is safe or bug-free. Everything depends on the private key staying secret.

02 How Windows checks the signature

When the file is opened, installed or inspected, Windows checks it through `WinVerifyTrust`. Every check must pass before Windows shows a verified publisher.

Checks, in order

1. **Hash matches**
   Windows recalculates the file's SHA-256 hash and compares it with the signed one.
2. **Chain reaches a trusted root**
   Your certificate → intermediate CA → a root in the machine's Trusted Root store.
3. **Certificate is for code signing**
   Its permitted use (Extended Key Usage) includes code signing, `1.3.6.1.5.5.7.3.3`.
4. **Valid at signing time**
   Checked against the timestamp. Without one, Windows uses today's date.
5. **Not revoked**
   Windows checks the CA's revocation lists (CRL or OCSP).

Trust chain

Root CA
Shipped by Microsoft's Root Certificate Program, or added by the client's IT through Group Policy

`signs`

Intermediate CA
e.g. the CA's code signing issuer

`signs`

Your Company Ltd
Code signing certificate; its key is held in your HSM

Then each layer applies its own policy

UAC prompt

Shows "Verified publisher: Your Company" for a valid signature. Otherwise it shows "Unknown publisher" in a yellow prompt.

SmartScreen

Checks downloaded files. Its decision is based on reputation, which builds up against your publisher identity over time. Since 2024, EV certificates no longer get instant reputation. It only checks files with a **Mark of the Web**: the hidden `Zone.Identifier` stream browsers and mail clients add. Files an installer extracts, or copies from a USB stick or some unzip tools, have none and aren't checked.

App Control (WDAC), AppLocker, Smart App Control

Corporate rules such as "allow only binaries signed by publisher X". This is how your clients' IT teams will allow-list you. Older environments may still use Software Restriction Policies; a block there, or by AppLocker, shows "This program is blocked by group policy".

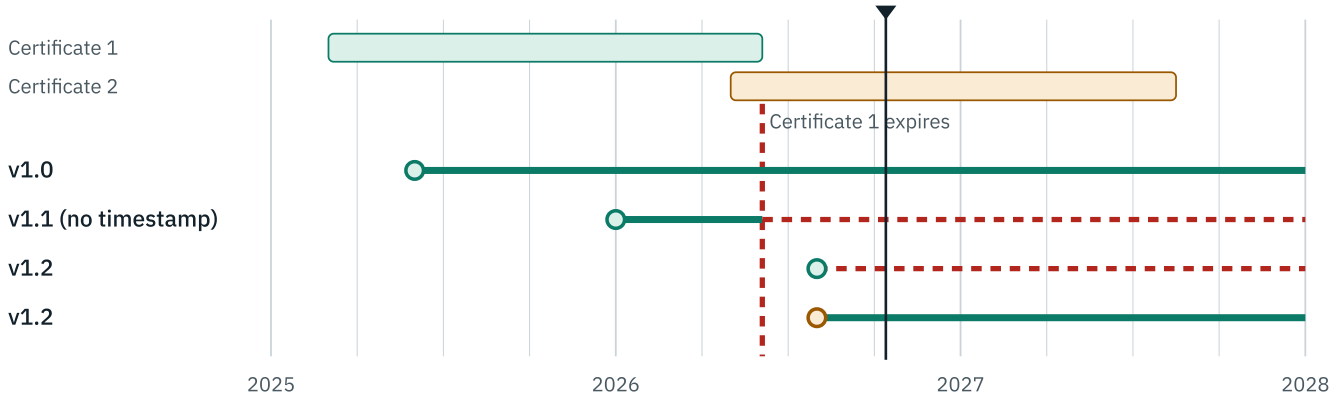
Defender and EDR

Take the signature and reputation into account when deciding whether to flag or quarantine a file.

```
Get-AuthenticodeSignature .\app.exe | Format-List *
signtool verify /pa /v app.exe
```

03 What happens when the certificate expires

It depends on whether the signature was **timestamped**. A timestamp from a trusted Timestamp Authority proves the file was signed while the certificate was valid, so the signature stays valid after expiry. Without one, the signature fails the day the certificate expires. Drag the date to see how four releases behave.



v1.0
Signed Jun 2025 · Certificate 1 · timestamped
Valid

v1.1
Signed Jan 2026 · Certificate 1 · no timestamp
Invalid: expired, no timestamp

v1.2 · old certificate
Signed Aug 2026 · Certificate 1 · timestamped
Invalid: certificate had expired

v1.2 · renewed certificate
Signed Aug 2026 · Certificate 2 · timestamped
Valid

Example dates. Certificate 1 runs from Mar 2025 to Jun 2026 (460 days). Certificate 2 is a renewal issued a month before Certificate 1 expires. Green lines mark periods when the signature is valid; red dashes mark periods when it is not.

- **Revocation is different from expiry.** If a certificate is revoked, signatures timestamped after the revocation date fail. If the key was compromised, the CA may backdate the revocation, which also breaks older releases.
- **Expired means you cannot sign.** No valid new signature can be made with an expired certificate. You need the renewed one.
- **Use SHA-256 throughout:** `signtool sign /fd sha256 /tr http://timestamp.digicert.com /td sha256 …`
- **Lifetime Signing EKU is the exception.** If the certificate carries it (`1.3.6.1.4.1.311.10.3.13`), signatures expire with the certificate even when timestamped. Check before you buy or renew.
- **Test it before it happens.** Set a test VM's clock past the certificate's expiry date, then run `signtool verify /pa /v`, `Test-ExecutableSignature.ps1` (section 11) and the installer itself.

04 Getting a certificate and watching its expiry

Fits external clients

Public CA

DigiCert, Sectigo, GlobalSign, SSL.com. **OV** (Organization Validated) is standard; **EV** (Extended Validation) has stricter vetting, and some customers' procurement teams still ask for it. Expect the CA to check business registration and D-U-N-S, and to call back a verified phone number.

Simplest for CI

Microsoft Trusted Signing

Being renamed Azure Artifact Signing. Microsoft verifies your organization and holds the keys in its own HSMs. Certificates last about 3 days and are issued automatically, so there is almost no expiry to manage. Only organizations in certain countries are eligible, so check yours first.

Internal only

Enterprise CA (AD CS)

Only trusted on machines that trust your internal root. Your clients' machines don't, so this does not suit software you ship to them.

Never sign production releases with a test (UAT) CA. Keep the CA's CRL/OCSP addresses reachable by every user, or revocation checks fail. If you also run an internal TSA, client PCs must trust its root as well, or the timestamp is ignored.

Making sure an expired certificate is never used

- **Gate in the pipeline.** Before signing, read the certificate's expiry date and fail the job if it has passed or is within N days. After signing, verify the signature and confirm a timestamp is present.
- **Scheduled GitLab pipeline.** Run the same check daily and alert at 60, 30 and 7 days by email or Slack, or by opening an issue.
- **Platform events.** Azure Key Vault raises a "certificate near expiry" event, and AWS and GCP offer similar alerts. Send CA renewal emails to a shared mailbox, not one person's inbox.
- **Ownership.** Track the certificate in your inventory or lifecycle tool (Venafi/CyberArk, Keyfactor, DigiCert Trust Lifecycle Manager). Name an owner and start renewal 4 to 6 weeks ahead, because the CA has to re-validate your organization.

```
# check-cert-expiry.sh: refuse to sign within 14 days of expiry
EXP=$(az keyvault certificate show --vault-name "$KV" -n "$CERT" --query attributes.expires -o
tsv)
DAYS=$(( ($(date -d "$EXP" +%s) - $(date +%s)) / 86400 ))
[ "$DAYS" -lt 14 ] && { echo "Certificate expires in $DAYS days - refusing to sign"; exit 1; }
```

05 What the client sees

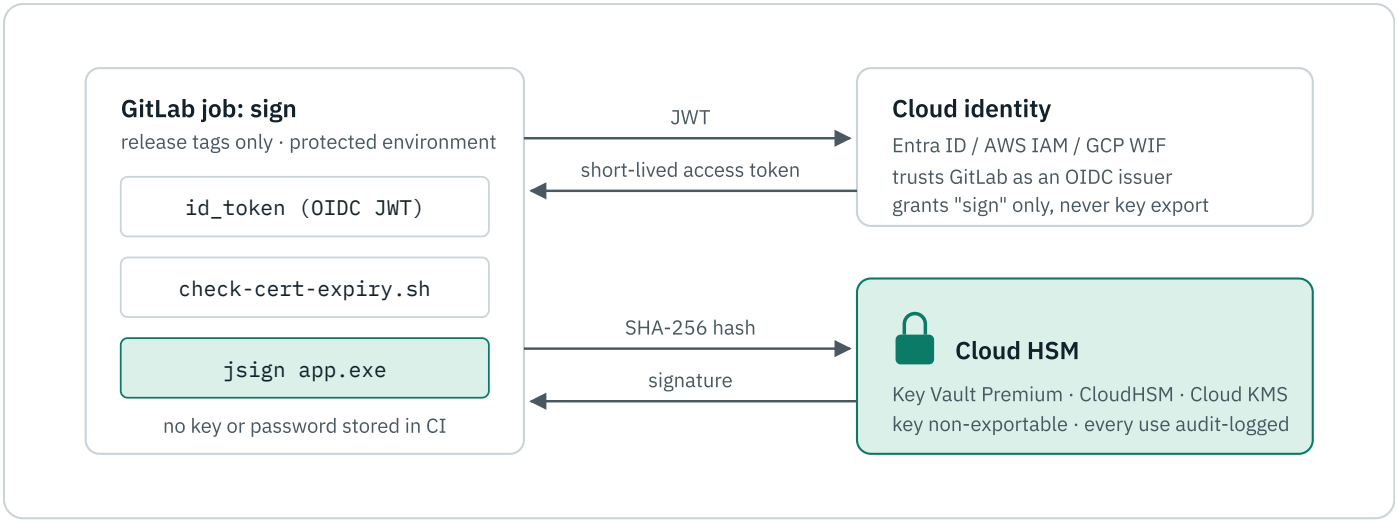
ARTIFACT	STANDARD WINDOWS	MANAGED CORPORATE ESTATE
Signed and timestamped; certificate expired later	Valid Verified publisher	Allowed by publisher rules
Signed without a timestamp; certificate now expired	Invalid "A required certificate is not within its validity period". A double-click still runs it, but UAC shows "Unknown publisher" and **Smart App Control blocks it**.	App Control (WDAC) publisher rules **usually still match**, unless the policy sets option 20. Signature checks in updaters fail, and EDR treats it as unsigned; it can look like tampering.
Signed with an already-expired certificate	Invalid Signature fails verification	Blocked, and likely flagged by security tools
Signed; certificate later revoked	Invalid for signatures after the revocation date; Smart App Control blocks them	Smart App Control and EDR react at once. App Control treats it as unsigned when option 20 is set.
Unsigned	Unknown publisher SmartScreen "Windows protected your PC"; Edge may warn on download	Usually blocked by App Control or Smart App Control, and rejected at security intake

- **App Control option 20** (`Enabled:Revoked Expired As Unsigned`) treats files signed with a revoked certificate, or with an expired certificate that carries the *Lifetime Signing* EKU (`1.3.6.1.4.1.311.10.3.13`), as unsigned. Without it, App Control doesn't reject a signature just because the certificate expired.
- **Lifetime Signing EKU:** signatures made with such a certificate expire with it **even when timestamped**. Code-signing certificates normally carry only the Code Signing EKU.
- Component by component (UAC policy, SmartScreen, AppLocker, EDR, your own update code): see Runtime impact of a missing timestamp (in *Test-ExecutableSignature.pdf*).

06 Keeping the key in an HSM and signing from GitLab CI

An HSM is required, not optional. Since June 2023, CA/Browser Forum rules require the private key of any publicly trusted code signing certificate to be generated and stored in certified hardware (FIPS 140-2 Level 2+ or Common Criteria EAL4+). CAs no longer issue downloadable `.pfx` files. The pipeline sends the hash to the HSM and gets a signature back. The key never leaves the device.

Build	Check expiry	Sign	Verify	Publish
compile on a protected tag	fail if expiring soon	OIDC → HSM, with timestamp	timestamp and full chain present	HTTPS, with SHA-256 hashes



The job proves its identity with a short-lived token GitLab issues for that run. It never handles the private key, only a hash going out and a signature coming back.

Ways to connect GitLab to an HSM

OPTION	HOW GITLAB CONNECTS	TRADE-OFF
Microsoft Trusted Signing	`signtool` with Microsoft's signing plug-in, or `jsign`; authentication through GitLab OIDC → Entra ID	Simplest. No HSM or certificate renewals to manage.
CA cloud signing DigiCert KeyLocker, SSL.com eSigner, GlobalSign, Sectigo	The CA's command-line tool (e.g. `smctl`) or `jsign`, with an API key in a protected, masked CI variable	Key sits in the CA's HSM. Priced per signature or per year.
Your own cloud HSM/KMS Azure Key Vault Premium / Managed HSM, AWS CloudHSM / KMS, Google Cloud KMS (HSM)	Generate the key and certificate request in the KMS, get them certified by a CA, then sign with `jsign` or `AzureSignTool` using OIDC	Full control. Confirm first that your CA accepts that HSM's key attestation.
USB token or on-premises HSM	Attached to a dedicated, protected self-hosted Windows runner; `signtool` or PKCS#11	Works, but fragile: PIN prompts, one physical machine, hard to scale.

Example signing job (Azure Key Vault and GitLab OIDC)

`jsign` runs on Linux runners (it is Java) and supports Azure Key Vault, Trusted Signing, AWS, GCP, DigiCert and PKCS#11 HSMs. You don't need a Windows runner just to sign.

```
sign:
  stage: sign
  image: your-registry/jsign-azcli:latest
  id_tokens:
    AZURE_JWT:
      aud: api://AzureADTokenExchange
  rules:
    - if: $CI_COMMIT_TAG                # release tags only
  environment: code-signing             # protected environment + approvers
  script:
    - az login --service-principal -u "$AZ_CLIENT_ID" -t "$AZ_TENANT_ID" --federated-token
"$AZURE_JWT"
    - ./check-cert-expiry.sh            # fails if expired or close to expiry
    - sh ci/check-tsa.sh "$TSA_URLS"    # fails if no TSA answers (section 07)
    - TOKEN=$(az account get-access-token --resource https://vault.azure.net --query accessToken
-o tsv)
    - jsign --storetype AZUREKEYVAULT --keystore "$KV" --storepass "$TOKEN" --alias "$CERT"
        --certfile "$CHAIN_PEM"
        --tsaurl "$TSA_URLS" --tsmode RFC3161 --tsretries 5 --tsretrywait 15
        --alg SHA-256 dist/app.exe
    - sh ci/verify-signature.sh dist/app.exe   # fails without timestamp or full chain
  variables:
    TSA_URLS: "http://timestamp.digicert.com,http://timestamp.sectigo.com"
  artifacts:
    paths: [dist/app.exe]
```

- **`--certfile` embeds the chain.** Cloud key stores and HSMs often give jsign only your certificate. Without the intermediate CA in the signature, the file validates only on PCs that already have that intermediate, and shows an "incomplete chain" everywhere else. `CHAIN_PEM` is a File-type CI variable with your certificate plus the intermediates.
- **`osslsigncode verify` isn't enough as a gate.** It reports a missing timestamp but doesn't fail on it. `verify-signature.sh` (section 07) reads the signature and fails explicitly.

Hardening checklist

- [ ] Sign only from protected branches or tags. Never sign in merge request pipelines, and especially not from forks.
- [ ] Put signing behind a protected environment with required approvers, on protected runners. Anyone who can trigger signing can sign malware with your company's name.
- [ ] Use GitLab OIDC (`id_tokens`) instead of long-lived secrets. Give the signing identity permission to sign and nothing else.
- [ ] Sign only what this pipeline built: build, then sign, then verify. Keep HSM and cloud audit logs so every signature traces back to a pipeline run.
- [ ] Sign everything you ship: the executable, every DLL, and the MSI or installer. Always timestamp.
- [ ] Sign PowerShell scripts you ship as well. Files an installer extracts carry no Mark of the Web, so `RemoteSigned` won't stop an unsigned or altered script.
- [ ] Embed the full certificate chain (`--certfile`), and verify the timestamp and chain on every signed file before publishing.

- [ ] Never change a file after signing it: compression (UPX), resource editing or re-packing breaks the signature. Sign inner files before embedding them, and the installer last (section 09).
- [ ] On the download site, serve over HTTPS and publish SHA-256 hashes next to each file.

If the signing key is compromised

1. **Stop signing.** Disable the signing identity and the protected environment so no further signatures can be made.
2. **Ask the CA to revoke** the certificate, giving the earliest date the key may have been misused. Releases timestamped *before* the revocation date stay valid; everything after it fails.
3. **Find out what was signed.** Go through the HSM and cloud audit logs and the pipeline history to list every signature made with the key.
4. **Get a new key and certificate** (new key in the HSM; never reuse the old one).
5. **Re-sign and timestamp every supported release** from trusted sources, re-publish them, and update the published SHA-256 hashes.
6. **Tell customers** which versions are affected, and give them the new certificate details so their IT teams can update allow-lists (section 10).

07 Timestamping in practice

A TSA URL in the pipeline configuration doesn't prove a file is timestamped. The timestamp exists only if the signing tool reached the TSA and stored its signed reply inside the signature, as an RFC 3161 token or a legacy countersignature. If that request was skipped or failed, there's no timestamp, whatever the configuration says. **Check the signed file.**

SETTING	USE	WHY
`--tsaurl`	Two or more URLs, comma-separated: `http://timestamp.digicert.com,http://timestamp.sectigo.com`	jsign tries them in order, so one TSA outage doesn't stop a release.
`--tsmode`	`RFC3161`	jsign defaults to the older `Authenticode` format. A TSA that only speaks RFC 3161 rejects that, which looks like a network error.
`--tsretries` / `--tsretrywait`	e.g. `5` / `15`	The defaults are 3 retries, 10 seconds apart.
jsign Maven plugin	Always set `<tsaurl>` and `<tsmode>`	Since jsign 7.6 the Maven plugin no longer timestamps by default, so an upgrade can silently drop timestamps.
`signtool`	`/tr <TSA> /td sha256`	`/t` is the legacy format.

Check the TSA before signing

Sends a real RFC 3161 request to each TSA and stops the job if none grants it. It contains only the hash of 32 random bytes, so nothing about your build leaves the runner.

```
#!/bin/sh
# ci/check-tsa.sh "http://tsa1,http://tsa2"
set -u
ok=""
for url in $(echo "$1" | tr ',' ' '); do
  head -c 32 /dev/urandom > /tmp/probe.bin
  openssl ts -query -data /tmp/probe.bin -sha256 -cert -out /tmp/probe.tsq
  if curl -fsS --max-time 15 ${PROXY_URL:+--proxy "$PROXY_URL"} \
       -H "Content-Type: application/timestamp-query" \
       --data-binary @/tmp/probe.tsq -o /tmp/probe.tsr "$url" \
     && openssl ts -reply -in /tmp/probe.tsr -text | grep -q 'Status: Granted'; then
    echo "TSA OK:     $url"; ok=1
  else
    echo "TSA FAILED: $url"
  fi
done
[ -n "$ok" ] || { echo "No timestamp authority reachable - refusing to sign."; exit 1; }
```

Verify every signed file

Reads the signature that ended up in each file. It fails if there's no timestamp (RFC 3161 OID `1.3.6.1.4.1.311.3.3.1`, or a legacy `countersignature`) or if only one certificate is embedded. Tested against Git for Windows (RFC 3161, 4 certificates), Chrome (legacy, 5 certificates) and an untimestamped file with 1 certificate, which it rejected.

```
#!/bin/sh
# ci/verify-signature.sh <file.exe> [<file2.exe> ...]
set -eu
[ "$#" -gt 0 ] || { echo "No files to verify"; exit 1; }
for f in "$@"; do
  osslsigncode extract-signature -pem -in "$f" -out /tmp/sig.pem \
    || { echo "FAIL: $f is not signed"; exit 1; }
  if openssl pkcs7 -in /tmp/sig.pem -print | grep -Eq
'1\.3\.6\.1\.4\.1\.311\.3\.3\.1|countersignature'; then
    echo "OK:   $f is timestamped"
  else
    echo "FAIL: $f has no timestamp"; exit 1
  fi
  n=$(openssl pkcs7 -in /tmp/sig.pem -print_certs | grep -c '^subject=')
  [ "$n" -ge 2 ] || { echo "FAIL: $f has only $n certificate - pass the full chain with --
certfile"; exit 1; }
  echo "OK:   $f has $n certificates embedded"
done
```

- **Never fall back to signing without a timestamp.** That file works until the certificate expires, then fails on every PC. Let the job fail instead.
- **Add a timestamp later**, without re-signing, while the certificate is still valid: `jsign timestamp --tsaurl … --tsmode RFC3161 app.exe` (jsign 7.0+) or `signtool timestamp /tr … /td sha256 app.exe`. After expiry only re-signing helps, because a timestamp would prove the signing happened after expiry.

08 When the TSA can't be reached

Timestamping is the only step in signing that has to reach a server outside your network. When it fails, check these, roughly from the most to the least common cause:

WATCH FOR	WHY IT HAPPENS	WHAT TO DO
Outbound port 80 blocked	Most public TSAs use plain HTTP. Many runner networks only allow 443. HTTP is fine here: the TSA's reply is signed, so it can't be tampered with in transit.	Allow outbound HTTP to the TSA hosts, or use a TSA that offers HTTPS.
Java ignores the proxy variables	Runners often set `HTTP_PROXY` / `NO_PROXY`. `curl` uses them; **Java, and so jsign, does not**. The TSA check can pass while jsign then fails.	Pass `--proxyUrl` (and `--proxyUser` / `--proxyPass`), or set `JAVA_TOOL_OPTIONS="-Dhttp.proxyHost=… -Dhttps.proxyHost=…"`. The Maven plugin takes the proxy from `settings.xml`.
TLS inspection on an HTTPS TSA	A corporate proxy re-signs HTTPS traffic with a CA Java doesn't trust: `PKIX path building failed`.	Add the proxy's root CA to the image's Java trust store (`keytool -importcert -cacerts …`), or use the HTTP URL.
Wrong `--tsmode`	jsign defaults to Authenticode; an RFC 3161-only TSA rejects it.	Always pass `--tsmode RFC3161`.

WATCH FOR	WHY IT HAPPENS	WHAT TO DO
DNS	Isolated runners may not resolve public names.	`getent hosts timestamp.digicert.com` in the job shows it at once.
Throttling	Public TSAs limit bursts, for example many parallel jobs each signing one file.	Sign all files in one jsign call, keep retries, and list a second TSA as a fallback.
Job timeout	Each failing URL costs up to retries × wait seconds before the next one is tried.	Size the job's `timeout:` for that, or use fewer retries with several URLs.
Internal TSA not trusted by clients	The request succeeds, but customers' PCs don't trust the TSA's certificate, so Windows ignores the timestamp.	Use a public TSA for software you ship to customers.

09 Several executables in one installer

When an installer carries other programs (Go `go:embed`, an MSI, a self-extracting archive), there are two layers of signatures, and the order is fixed: **inner files first, installer last**. Embedding copies the bytes at build time, so a file signed after embedding stays unsigned inside the installer. Signing the installer changes nothing inside it; it adds the outer signature over everything.

Build
agent.exe, updater.exe, ...

Sign each
inner signatures, timestamped

Verify each
stop before embedding if any fails

Embed
go build the installer

Sign installer
outer signature, then verify

Extracted files have no Mark of the Web

The downloaded installer has a Mark of the Web; the programs it writes don't. Windows doesn't pass the mark on to files a program creates (tested with a Go installer). That's normal for all installers, and it changes which checks apply:

CHECK	DOWNLOADED INSTALLER	EXTRACTED PROGRAMS
SmartScreen reputation	Checked	Never checked
Smart App Control, App Control (WDAC), AppLocker	Checked	Checked on every launch; they don't depend on the mark
Defender / EDR	Scanned	Scanned, like any installed software
UAC prompt	Publisher from the signature	Publisher from the signature

- **The inner signatures carry the trust after installation.** Sign and timestamp every embedded exe and DLL. After the certificate expires, an untimestamped inner exe is blocked by Smart App Control and fails your updater's signature checks.
- **Extract to Program Files**, not `%TEMP%` or Downloads. Nothing will warn later if someone replaces an extracted file or plants a DLL next to it.

- **Don't add the mark to installed files yourself.** Every launch would trigger SmartScreen, and users learn to click through warnings.
- In managed environments, IT can make the installer a **WDAC managed installer**: every file it writes is then tagged and allowed automatically.

Maven: phases in the right order

Plugins bound to the same Maven phase run in the order they're declared, so give each step its own phase and declare `jsign-maven-plugin` before `exec-maven-plugin`. One execution with a `<fileset>` signs all inner files.

PHASE	PLUGIN	STEP
`clean`	clean	Empty the payload folder, so a stale unsigned exe can never be embedded
`initialize`	exec	`check-tsa.sh`
`compile`	exec	`go build -o installer/payload/ ./cmd/...`
`prepare-package`	jsign	Sign every file in the payload folder
`package`	exec	Verify the payload, then `go build` the installer
`verify`	jsign, then exec	Sign the installer, then verify it

```
<execution>
  <id>sign-payload</id>
  <phase>prepare-package</phase>
  <goals><goal>sign</goal></goals>
  <configuration>
    <fileset>
      <directory>${payload.dir}</directory>
      <includes><include>*.exe</include><include>*.dll</include></includes>
    </fileset>
  </configuration>
</execution>
<!-- shared plugin configuration: <algorithm>SHA-256</algorithm>, <certfile>, <tsaurl>,
     <tsmode>RFC3161</tsmode>, <storepass>mvn:code-signing</storepass> (password from
settings.xml) -->
```

The jsign Maven plugin has no proxy settings of its own; it uses the proxy in Maven's `settings.xml`. The full `pom.xml`, the installer code, `settings.xml` and the GitLab job are in Test-ExecutableSignature.pdf.

10 Version info and allow-listing

Your customers' IT teams allow your software with App Control (WDAC) or AppLocker rules. Those rules match more than the certificate: they also use fields from the version information compiled into each file. Keep those fields stable, or every release breaks your customers' rules.

FIELD	USED BY	KEEP IT
Certificate subject (your organization)	WDAC Publisher and FilePublisher rules; AppLocker publisher rules	The same legal name across renewals

FIELD	USED BY	KEEP IT
Issuing CA (intermediate)	WDAC Publisher and FilePublisher rules	Changing CA means customers must update their rules; tell them in advance
`OriginalFilename`	WDAC FilePublisher rules; AppLocker file name	Stable for each program; never reused for a different program
`FileVersion`	WDAC FilePublisher minimum version; AppLocker version range	Always increasing
`ProductName`	AppLocker publisher rules	Stable
`CompanyName`	Shown to users; compared with the signer by `Test-ExecutableSignature.ps1`	The organization in your certificate

An allow-listing note for customers

Publish this with each major release, so customers' IT teams don't have to work it out themselves:

- Certificate subject, issuing CA and root CA, and the thumbprint of the current certificate and its expiry date.
- The `OriginalFilename` of every program you ship, and the first version covered.
- Whether the installer is suitable as a WDAC **managed installer**.
- Ready-made rule commands:

```
# App Control (WDAC) rule: publisher + file name + minimum version, hash as a fallback
# (needs the ConfigCI module, which not every Windows edition has)
New-CIPolicyRule -DriverFilePath .\app.exe -Level FilePublisher -Fallback Hash

# AppLocker publisher rule
Get-AppLockerFileInformation -Path .\app.exe |
  New-AppLockerPolicy -RuleType Publisher -User Everyone -Xml
```

11 Checking a release on Windows

Three PowerShell scripts in this repository check a file without running it. Use them as a release gate on a Windows runner, and point customers to them.

SCRIPT	CHECKS	WHO ACTS ON IT
`Test-ExecutableSignature.ps1`	Signature, certificate, timestamp, chain, revocation, publisher identity. Prints the days left until an untimestamped signature expires.	The publisher: every finding comes with the fix
`Test-ExecutableEnvironment.ps1`	Hash, Mark of the Web, Defender scan, App Control and Smart App Control prediction, Group Policy blocks (Software Restriction Policies, AppLocker events), permissions	The user, or their IT team
`Test-ExecutableSafety.ps1`	Both together, in one report	-

As a release gate

```
release-check:
  stage: verify
  tags: [windows]
  needs: [sign]
  script:
    - powershell -ExecutionPolicy Bypass -File .\Test-ExecutableSignature.ps1
        -Path dist\app.exe -ExpectedPublisher "Your Company Ltd" -ReportPath signature-
report.json
  allow_failure:
    exit_codes: [1]          # 1 = warnings; 2 = failures stop the release
  artifacts:
    when: always
    paths: [signature-report.json]
```

- Exit code `2` (a broken, untrusted, untimestamped-and-expired or revoked signature) fails the job. Exit code `1` lets it pass with a warning.
- Check the installer **and** each embedded program: run the installer into a test folder first.
- For customers who report a blocked install, `Test-ExecutableEnvironment.ps1` shows whether App Control, Smart App Control, Group Policy (Software Restriction Policies or AppLocker: "This program is blocked by group policy") or Defender is the cause, and what their IT team can do.

Reflects CA/Browser Forum rules, Microsoft behaviour and jsign 7.6 as of November 2026. Confirm the specifics with your CA before relying on them: maximum validity, Trusted Signing eligibility and accepted HSMs all change.